Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования

Национальный исследовательский Нижегородский государственный университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

Множества на основе битовых полей

Выполнил: студент группы _____

Игнатенков А. _____

Проверил: Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение....3

1 Постановка задачи....3

2 Руководство пользователя....4

3 Руководство программиста....5

3.1 Описание структуры программы....5

3.2 Описание структур данных....5

3.3 Описание алгоритмов....6

4 Результаты экспериментов и тестирования....7

Заключение....8

Приложение....9

Введение

Множество хранит элементы без повторений. Для множества целых чисел из ограниченного диапазона удобно использовать битовое поле: каждому возможному элементу соответствует один бит. Если бит равен 1, элемент входит в множество; если 0, элемент отсутствует.

Например, множество {1, 3, 5} в универсуме от 0 до 7 представляется последовательностью 01010100, если позиции перечислять от индекса 0 к индексу 7. Такая запись является характеристическим вектором множества.

Цель лабораторной работы — реализовать классы TBitField и TSet и проверить их работу с помощью Google Test. В отчете описаны хранение данных, основные алгоритмы и результаты тестирования.

1 Постановка задачи

Необходимо реализовать методы двух классов по выданным заголовочным файлам. Объявления классов должны остаться неизменными. Реализация размещается в отдельных файлах tbitfield.cpp и tset.cpp.

Класс TBitField должен поддерживать создание и копирование поля, доступ к отдельным битам, присваивание, сравнение, побитовые операции ИЛИ, И и НЕ, а также потоковый ввод и вывод. Память для битов выделяется динамически.

Класс TSet использует TBitField для хранения элементов. Требуется реализовать включение и удаление элемента, проверку принадлежности, объединение, пересечение, дополнение, сравнение, присваивание и преобразование к битовому полю.

Объединение и пересечение множеств допускаются только при одинаковом размере универсума. В реализации этому соответствует равенство MaxPower. Размер универсума отличается от числа вставленных элементов: TSet(8) задает допустимые элементы от 0 до 7 независимо от заполнения.

Отдельная задача — подготовить тесты для обычных операций и граничных случаев: пустых объектов, неправильных индексов, независимости копий и ошибок ввода.

2 Руководство пользователя

Результат работы представляет собой библиотеку классов и набор автоматических тестов. Отдельного консольного меню в проекте нет. Классы используются из программы или тестов через заголовки tbitfield.h и tset.h.

Для сборки нужен компилятор C++ и подключенный Google Test. Файлы реализации и оба файла тестов включаются в тестовый проект. Заголовок gtest/gtest.h должен находиться в пути поиска компилятора. При использовании gtest_main собственная функция main не требуется.

В GCC при установленной библиотеке и настроенных путях поиска можно выполнить следующие команды из папки Lab1:

```
g++ -std=c++14 tbitfield.cpp tset.cpp \
    test_tbitfield.cpp test_tset.cpp \
    -lgtest_main -lgtest -pthread -o lab1_tests
./lab1_tests
```

Для создания множества указывается размер универсума. В следующем примере оба множества имеют универсум от 0 до 7:

```
TSet a(8), b(8);
a.InsElem(1);
a.InsElem(3);
b.InsElem(3);
b.InsElem(5);
TSet united = a + b;
TSet common = a * b;
TSet other = ~a;
```

Результаты: united содержит {1 3 5}, common содержит {3}, other содержит {0 2 4 5 6 7}. Методы DelElem и IsMember удаляют элемент и проверяют его наличие. Выражения a + 2 и a - 1 возвращают измененные копии множества a.

Множество вводится в формате {1 3 5}, элементы разделяются пробелами. Пустое множество записывается как {}. Битовое поле вводится строкой из нулей и единиц точно заданной длины; первый символ соответствует биту №0. Ввод заменяет прежнее содержимое объекта.

Отрицательная длина и разные универсумы при объединении или пересечении вызывают invalid_argument. Индекс вне допустимого диапазона вызывает out_of_range. При неверном формате ввода устанавливается failbit, прежнее содержимое объекта сохраняется.

3 Руководство программиста

3.1 Описание структуры программы

Программа состоит из двух классов и двух файлов тестов. Заголовки содержат исходные объявления преподавателя, а определения методов вынесены в файлы реализации.

Файл	Назначение
tbitfield.h	Объявление TBitField и типа TELEM
tbitfield.cpp	Работа с памятью, битами и операторами поля
tset.h	Объявление TSet
tset.cpp	Операции с множествами через битовое поле
test_tbitfield.cpp	22 теста битового поля
test_tset.cpp	29 тестов множества

3.2 Описание структур данных

TELEM — другое имя типа unsigned int. Константа bitsPerElem вычисляется как sizeof(TELEM) * CHAR_BIT и задает количество битов в одном слове массива. При 32-битном unsigned int для хранения 70 бит нужны три слова.

Поле	Смысл
BitLen	Логическая длина битового поля
pMem	Указатель на динамический массив TELEM
MemLen	Количество слов в массиве pMem
MaxPower	Размер универсума множества
BitField	Объект TBitField внутри TSet

Массив создается оператором new[] и заполняется нулями. При копировании выделяется собственный массив, поэтому изменение копии не затрагивает оригинал. Деструктор освобождает память через delete[]. Поле длины 0 допустимо; доступных индексов в нем нет.

3.3 Описание алгоритмов

Для бита с номером n сначала вычисляются номер слова и маска. В методе GetMemIndex используется целочисленное деление n / bitsPerElem. Метод GetMemMask возвращает TELEM(1) << (n % bitsPerElem). Например, при 32 битах в слове бит №35 находится в pMem[1] на позиции 3, а его маска равна 8.

SetBit применяет ИЛИ с маской: pMem[index] |= mask. ClrBit очищает бит выражением pMem[index] &= ~mask. GetBit проверяет, отличается ли результат pMem[index] & mask от нуля. Перед обращением к массиву проверяется условие 0 <= n < BitLen.

При присваивании сначала проверяется самоприсваивание. Затем создается новый массив и в него копируются слова второго объекта. После этого старый массив удаляется и обновляются указатель и размеры. Такое расположение действий сохраняет старый объект, если выделить новый массив не удалось.

Операторы | и & работают со словами массива. Результат имеет большую из двух длин; отсутствующие слова короткого поля считаются нулевыми. Это правило относится к TBitField. В TSet до выполнения объединения или пересечения проверяется равенство MaxPower, поэтому разные универсумы не допускаются.

Дополнение TBitField реализовано проходом по логическим битам: если исходный бит равен 0, в результате он устанавливается в 1. Биты за границей BitLen не затрагиваются. Дополнение TSet выполняется в пределах текущего универсума.

Методы TSet вызывают соответствующие методы BitField. InsElem устанавливает бит, DelElem очищает его, IsMember читает значение. Объединение использует |, пересечение — &, дополнение — ~. Операции с отдельным элементом сначала создают копию множества.

При вводе данные записываются во временный объект. Если весь ввод корректен, он присваивается целевому объекту. При ошибке поток получает failbit, а исходные данные остаются прежними.

Чтение, установка и очистка отдельного бита требуют O(1) времени. Копирование и побитовые ИЛИ и И требуют O(M), где M — число слов массива. Дополнение в этой реализации, а также вывод поля и множества требуют O(N), где N — длина поля. Объем памяти для хранения равен O(M).

4 Результаты экспериментов и тестирования

Проверка выполнена с Google Test 1.15.2 и компилятором GCC 13.3.0 в Linux. Использован стандарт C++14. Сборка проверялась с предупреждениями -Wall -Wextra -Wpedantic и -Werror, а запуск — с UndefinedBehaviorSanitizer. Сообщений об ошибках неопределенного поведения не получено.

В наборе 51 тест: 22 для TBitField и 29 для TSet. Каждый тест создает объекты, выполняет проверяемую операцию и сравнивает фактические значения с ожидаемыми. Ожидаемые исключения проверяются через EXPECT_THROW.

Проверяемая группа	**Примеры случаев**
Создание объектов	Нулевая, положительная и отрицательная длина
Доступ к битам и элементам	Установка, очистка, повторы, неверные индексы
Границы хранения	Биты до и после границы машинного слова
Копирование и присваивание	Независимость копий, изменение размера
Операции	Объединение, пересечение и дополнение
Преобразования	Из битового поля в множество и обратно
Потоковый ввод и вывод	Формат записи, замена содержимого, ошибки
Универсум	Разные размеры, пустые множества и нулевой размер

После замечания преподавателя обновлены тесты объединения и пересечения: обычные операции используют одинаковые MaxPower. Дополнительные тесты ожидают исключение при разных универсумах, в том числе если множества не содержат элементов. Два множества с универсумом размера 0 совместимы.

Итог запуска Google Test:

```
[==========] 51 tests from 2 test suites ran.
[  PASSED  ] 51 tests.
```

Все 51 тест прошли. Проверка подтверждает работу реализованных операций на рассмотренных примерах и граничных случаях; она не является измерением быстродействия или проверкой всех возможных входных данных.

Заключение

Реализованы TBitField и TSet с сохранением исходных объявлений классов. Битовое поле хранится в динамическом массиве беззнаковых целых чисел. Для доступа к одному биту используются индекс слова и битовая маска.

Поддерживаются копирование, присваивание, сравнение, операции над битами и множествами, преобразования типов и потоковый ввод и вывод. После исправления объединение и пересечение множеств разрешены только при равных размерах универсума.

Подготовлен 51 автоматический тест Google Test. При проверке все тесты прошли. В работе использованы побитовые операции C++, динамическое выделение памяти, конструктор копирования и перегрузка операторов.

Приложение

Фрагмент tset.cpp: проверка совпадения универсумов.

```
TSet TSet::operator+(const TSet& s) {
    if (MaxPower != s.MaxPower) {
        throw std::invalid_argument(
            "Different set universes");
    }
    return TSet(BitField | s.BitField);
}
```

Фрагмент test_tset.cpp: проверка исключений для пересечения.

```
TEST(Set, IntersectionOfDifferentUniverses) {
    TSet first(5);
    TSet second(8);
    first.InsElem(2);
    second.InsElem(2);

    EXPECT_THROW(first * second,
                 std::invalid_argument);
    EXPECT_THROW(second * first,
                 std::invalid_argument);
    EXPECT_EQ(first.IsMember(2), 1);
    EXPECT_EQ(second.IsMember(2), 1);
}
```

Первый фрагмент запрещает объединение разных универсумов до выполнения побитовой операции. Второй проверяет отказ в обоих порядках операндов и сохранение содержимого множеств. Аналогичная проверка добавлена для объединения.